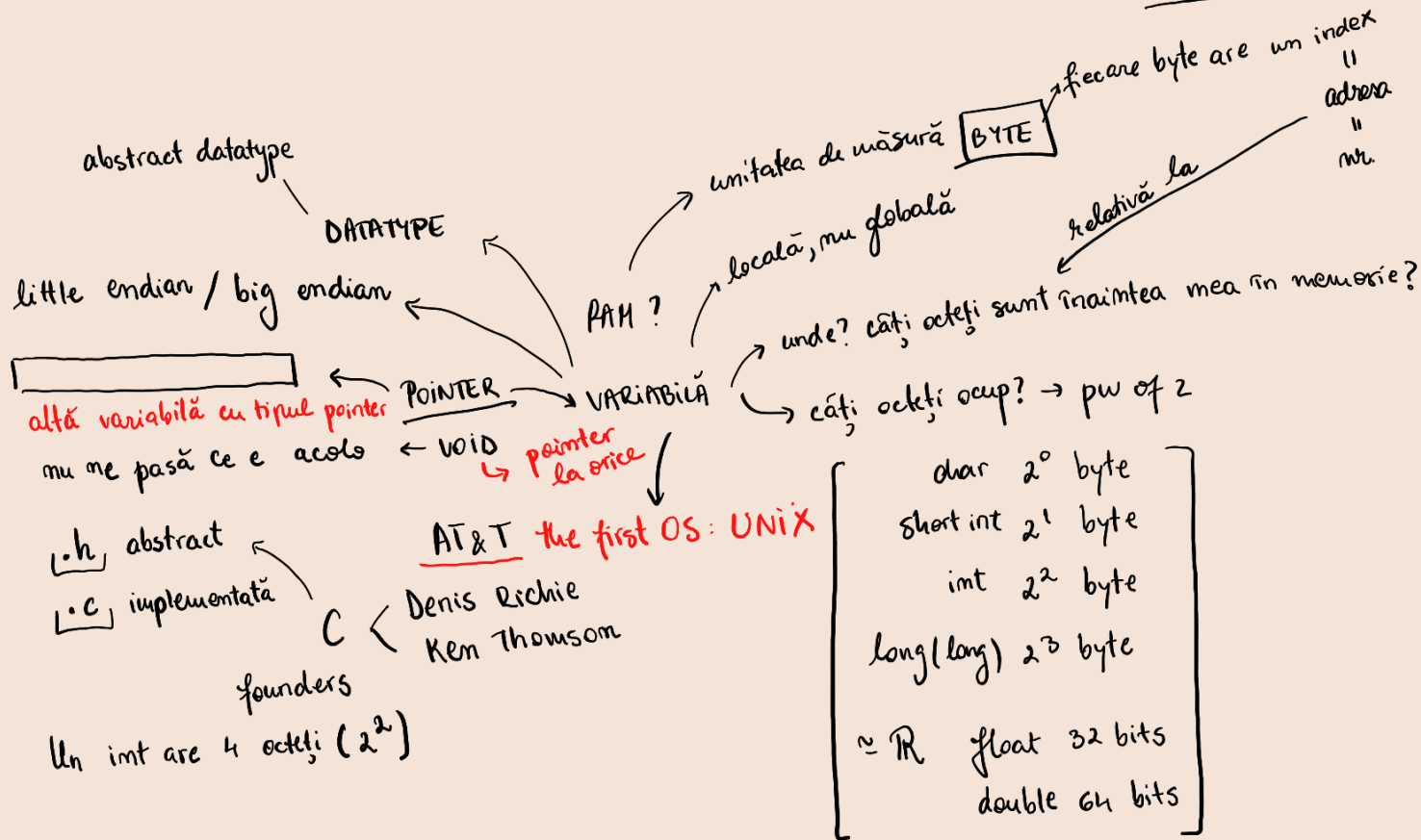


24.10.2025



POINTER: O zonă de memorie care conține în interiorul ei o variabilă care este interpretată la ^{altă} adresă de memorie.

- număr - labelul asigurat unei adrese
- facem o variabilă locală. NU GLOBALĂ.
- dăm denumiri SUGESTIVE

→ la printf mereu încheiem cu \n sau \o, altfel nu dă flash.

int* char* void* A* → au aceeași dimensiune

↓
depinde de procesor. $\begin{cases} X32 \\ X64 \end{cases}$

! Pointer ARE adresa.

↳ are în el un nr care e de fapt o adresă)

! Totdeauna inițializăm variabilele.

int* val = &nr; (⇒) int *val = ...

↓
"pointer la int"

↓
și luăm adresa de unde se află în memorie

~~int a;~~

int main()

{

int numar = 247777216;

void* pointer_la_numar = &numar // → 0x7ffecfb29174 în hexa

Char* pointer_la_octetul_0 = &numar

Char* pointer_la_octetul_1 = pointer_la_octetul_0 + 1

Char* pointer_la_octetul_2 = pointer_la_octetul_1 + 1

Char* pointer_la_octetul_3 = pointer_la_octetul_2 + 1

printf("%p\n", *pointer_la_numar)

printf("format: \"%p\n\", &numar)

*(pointer_la_octetul_1)++

*(pointer_la_octetul_3)--

tipul de
dată
"pointer
la
void"



} return 0;

adresa 0x7f580

→ instanță oice
* void
↓
ia oricât

Explicit: mergi la adresă
↓
cât e tipul pointerului?

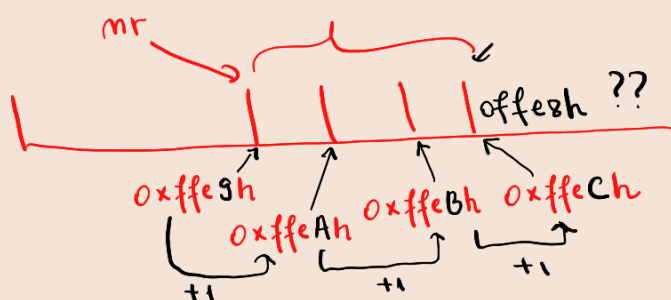
* mr
↓
dereferențiere a pointerului mr.
↓
vede adresa
↓
ia de acolo 4 octeți (2² byt)
↓
int.
↓
interpretează

int mr = 7 (valoare)

int * pointer_la_mr = 8 mr → "se schimbă tipul de dată"

char * pointer_la_octetul_0 = 8 mr

char * pointer_la_octetul_1 = pointer_la_octetul_0 + 1



pointer_la_mr (label)

adresa + câți-octeți-are-un int } crește valoarea
+ 4

adresa + 1 } crește adresa

Pointer la int, → ie 4 octeți ⇒ convert la cu mai mic
Pointer la char, → ie 1 octet

→ me ducem în int care e little endian

→ avem 7 0 0 0 → pointer la char

octet0 octet1 octet2 octet3

↓ dereferențiem

char * pointer la octetul 1

||
char

↓
adresa la octet octet

adresa

octet

11111111
octet 3

00000000

00000000

0000 111

unsigned char *

1. Ce sunt RAM-ii

RAM (Random Access Memory) este memoria volatilă a calculatorului, unde sunt stocate temporar datele și instrucțiunile în timpul execuției unui program.

Unitatea de măsură a memoriei este **byte-ul** (nu bitul).

Fiecare byte are un **index unic** — adică **o adresă de memorie**, care este un număr.

Adresa unui byte depinde de **câți octeți se află înaintea lui** în memorie.

2. Variabile, dimensiuni

O variabilă ocupă un număr fix de **octeți**, în funcție de tipul ei de date.

Aceste dimensiuni sunt puteri ale lui 2:

Tip de date	Dimensiune (octeți)	Formula
char	1	2^0
short int	2	2^1
int	4	2^2
long / float	4	2^2
long long / double	8	2^3
long double	12 / 16 (depinde de compilator)	—

3. Ce este un pointer

Un **pointer** este o zona de memorie care contine in interiorul ei o variabila care e interpretata la alta adresa de memorie. Deci o **variabilă specială** care conține **adresa de memorie** a unei alte variabile.

Un pointer nu este o adresă*, ci **are o adresă și conține o altă adresă**.

Numar - labelul asignat unei adrese

```
int nr = 10;  
int* p = &nr; // p conține adresa lui nr
```

4. Tipuri de pointeri

- `int*` → pointer la un `int` (ia 4 octeți din memorie)
- `char*` → pointer la un `char` (ia 1 octet)
- `void*` → pointer generic (poate pointa la orice tip, dar nu poate fi dereferențiat direct)
- `a*` (unde `a` e un tip de date) → pointer la acel tip.

Toți pointerii au **aceeași dimensiune**, care depinde de procesor:

- pe **sisteme x86 (32-bit)**: o adresă are 4 octeți (ex: `0x0040AF22`)
- pe **sisteme x64 (64-bit)**: o adresă are 8 octeți (ex: `0x00007FFDC3B2A890`)

Fiecare cifră hexa = **4 biți**

o adresă pe 32-bit are 8 cifre hexa ($4 \times 8 = 32$ biți)

o adresa pe 64-bit are 16 cifre hexa ($4 \times 16 = 64$ biți).

5. Little endian vs Big endian

Little endian - azi.

În memorie, octetul cel mai mic (least significant byte) e stocat primul (la adresa cea mai mică).

```
int x = 0x12345678;
Memorie: 78 56 34 12
```

Big endian = octetul cel mai mare e primul (folosit rar, ex. rețelistică).

6. Pointerii în acțiune

```
int main() {
    int numar = 247777216;
    void* pointer_la_numar = &numar; // adresa în memorie (în hexazecimal)

    // Alegem un pointer la char pentru a parcurge byte cu byte
    char* pointer_la_octetul_0 = (char*)&numar;
    char* pointer_la_octetul_1 = pointer_la_octetul_0 + 1;
    char* pointer_la_octetul_2 = pointer_la_octetul_1 + 1;
    char* pointer_la_octetul_3 = pointer_la_octetul_2 + 1;

    printf("%p\n", pointer_la_numar); // adresa lui numar
    printf("%p\n", &numar);          // identic cu linia de mai sus

    // Experimentăm modificarea fiecărui byte
```

```

(*pointer_la_octetul_1)++;
(*pointer_la_octetul_3)--;

return 0;
}

```

- `char` are 1 byte $\Rightarrow$ ne permite să vedem fiecare octet dintr-un `int`.
- `+1` pe pointer crește **adresa** cu 1 byte.
- `++` pe *valoare* modifică conținutul la adresa respectivă.
- `++` aplicat direct pe pointer (`pointer++`) mută adresa la următorul obiect de acel tip (ex. la `int*`, mută cu +4).

8. Exemple cu `char*` și `int*`

```

int nr = 7;
int* p_int = &nr;
char* p_char = (char*)&nr;
char* p_char2 = p_char + 1;

```

Memoria pentru `nr = 7` (pe little endian, 4 octeți):

```
07 00 00 00
```

- `p_char` $\rightarrow$ arată spre primul byte (07)
- `p_char + 1` $\rightarrow$ arată spre al doilea byte (00)
- `(*p_char2)++` $\Rightarrow$ 07 01 00 00
 $\rightarrow$ valoarea lui `nr` se schimbă

De ce uneori apare un număr mare negativ (ex. -16776953)

Dacă încercăm să afișăm valorile obținute din `char*` cu `%d`, ele pot apărea negative sau foarte mari.

Motivul e că un `char` e **signed** în multe compilatoare (variază între implementări).

Astfel, un byte cu valoarea `0xFF` (255 în `unsigned char`) este interpretat ca **-1** în `signed char`.

De exemplu, dacă un octet conține `0xFF`, `printf("%d", *p_char);` va afișa **-1**, iar dacă e extins greșit la `int`, pot vedea valori precum **-16776953** — nu e o adresă și nici valoarea reală, ci o conversie greșită de semn.

Pentru a afișa corect conținutul octeților:

```
printf("%u\n", (unsigned char)*p_char); // afisează 0-255 corect
printf("%p\n", (void*)p_char);        // afișează adresa reală în hex
```

9. Operatori pe pointeri

Operator	Ce face	Explicație
<code>++p</code>	crește adresa cu dimensiunea tipului (4 pentru <code>int*</code> , 1 pentru <code>char*</code>)	mută pointerul
<code>p + 1</code>	adresa + dimensiunea tipului	identic cu <code>++p</code>
<code>(*p)++</code>	crește valoarea de la acea adresă	modifică conținutul
<code>p[i]</code>	<code>*(p + i)</code>	accesează elementul i dintr-un tablou

10. Flash și Segmentation Fault

„**Flash**”: dacă nu închei cu `\n`, `printf` nu trimite imediat textul spre ecran — outputul e *buffered*.

`"\n"` golește bufferul, de aceea se vede rezultatul instant.

„**Segmentation fault**”: când un pointer accesează o zonă de memorie „**interzisă**” sau **neinițializată** (în afara zonei alocate programului).

11. void

`void` = „nimic” / „tip generic”.

- `void` singur → funcția nu returnează nimic.
- `void*` → pointer generic, poate ține adresa oricărui tip, dar trebuie convertit (cast) înainte de dereferențiere.

12. Conclusions

- RAM = zonă temporară de memorie, acces direct prin adrese.
- Adresele sunt numere în hex, dimensiunea depinde de arhitectura procesorului (x86 vs x64).
- Pointerul **ține o adresă**, nu o valoare.
- `*` (dereferențiere) → accesează valoarea la acea adresă.

- `&` (adresa) → obține adresa unei variabile.
- `char*` e util pentru vizualizarea byte cu byte.
- Endianness = ordinea de stocare a byte-ilor în memorie.
- `void*` e universal, dar trebuie convertit.
- `printf` fără `\n` poate să nu afișeze imediat.
- „Segmentation fault” = acces la o parte din memorie care nu e dedicată programului